

Legacy Application Management Cheat Sheet

Introduction

Legacy applications are applications that are recognized as being outdated but remain in active use by an organization. This may occur if a viable alternative to the application is not available, replacement is currently too expensive, or if the application is highly bespoke and services a very niche role within an organization's digital ecosystem. Legacy applications often introduce significant security risks to an organization for the following reasons:

- Legacy applications might have reached End-of-Life (EoL) meaning that the application no longer receives patching or vendor support. This drastically increases the risk of vulnerability in the application being left in an exploitable state.
- Some applications have been built using technologies that are no longer conventionally used or taught to technical staff. This can mean that the knowledge required to troubleshoot or fix vulnerabilities when they arise may be lacking.
- Legacy applications may produce data in custom formats and/or use old interfaces or networking protocols. This may stifle efforts to use data produced by the application with services used for vulnerability management or security logging, such as a SIEM (Security Information and Event Management) solution.

As there is no one-size fits all approach to securing legacy applications, this cheat sheet is intended to act as a resource offering some practical suggestions on hardening legacy applications when alternatives to the use of legacy applications do not exist. The preferred approach will depend on factors such as the type of data stored and produced by the legacy application, whether the application and associated infrastructure have known vulnerabilities, and the extent to which it is possible to restrict access to the legacy application or some of its riskier components. Engaging with security domain experts (in-house or external) who can provide specific and contextualized advice may be necessary.

Inventory and Asset Management

At a baseline, organizations should have a clear understanding about what legacy applications are currently in use and what the expected risk of the use of these legacy solutions are.

Inventory Management: Start by compiling documentation identifying the legacy applications used by your organization including version numbers, date of production, and relevant configuration settings. Ideally, this will include details regarding what network hosts need to be situated on to reach the application and associated infrastructure. A record of the services

running on infrastructure used for hosting the application and/or for data storage should also be outlined. In some circumstances documentation could include information about the physical location of and permitted access to servers associated with the application. Organizations might opt to generate a formal SBOM (Software Bill of Materials) as part of this process. SBOMs serve a useful role when an application relies on third-party dependencies in order to function.

Risk Assessment and Threat Modeling: Ensure your organization has a clear understanding of the level of risk and the kinds of threats theoretically posed by using the legacy application and its specific components (e.g. specific API routes or open ports on hosting infrastructure). This may be informed by formal or informal threat-modeling of an application, as described in the [Threat Modeling Cheat Sheet](#). Qualifying the risk posed by legacy software might also be aided by using an industry standard risk assessment framework, such as the NIST (National Institute of Standards and Technology) [Risk Management Framework](#). There are many threat modeling and risk assessment frameworks and tools that have different strengths and weaknesses.

As a more informal indicator of how conservative security measures used to protect the application ought to be, consider the questions below. These may help to contextualize what the risk profile of a particular legacy application or its components might be:

- What information is handled/stored by the application? If this information were to be compromised, how would this impact your business and potentially its regulatory/compliance requirements?
- Do the application/application dependencies/infrastructure used to support the application have known vulnerabilities? If so, how easily exploitable are these? Can these be patched with the right resources including skilled professionals?
- How critical is the availability of the legacy application to your organization's business continuity?
- If an attacker were able to gain access to this application, is there a risk that they could use it to exfiltrate other critical information from your organization? Could an attacker establish access to a particularly privileged network or environment by compromising the legacy application?

Authentication/Authorization

Authorization measures enforce rules around who can access a given resource and how they can establish access to that resource. Authorization is covered in significant depth in the [Authorization Cheat Sheet](#).

When it comes to applying authorization controls to legacy systems, organizations should consider the applications to be inherently high risk. The security principle of least privilege (allowing only as much access as is strictly required for staff/users/systems to perform their

required roles and to facilitate business operations) applies especially to legacy applications. Consider implementing the following as applicable:

- Apply network-level access controls to the application. This might entail hosting the application within a restricted subnet and/or applying IP allow-listing to prevent arbitrary users from interacting particularly with public-facing legacy applications from arbitrary hosts. In some circumstances the application may be required to run in an air-gapped environment.
- Authorization controls could be considered at a more granular level by reducing the feature set available to end users. For example, it might be necessary to disable certain high risk functionalities, particularly administrative functionalities.
- Ensure that only authenticated users can access the application. This could be enforced by the application itself, or by use of an IdP (Identity Provider) service. If the application is hosted in a restricted network environment, authentication should also be required to access this network environment (e.g. users could be required to authenticate to a VPN server before accessing the application). Implementing one or more of these controls will both slow down an attacker and assist with investigations if an incident were to occur. See the [Authentication Cheat Sheet](#) for further information regarding authentication controls.
- Close any ports on hosts used to run the application that are not strictly needed in order for the application to perform only the tasks required of it by your organization. Access to certain ports may also be restricted using firewall/application firewall rules to lock down server infrastructure.
- In some circumstances it may be possible to restrict almost all users from directly accessing the legacy application by developing an intermediary service (e.g. a separate set of APIs) that handles essential movement of data into and out of the legacy application without an end user having any requirement to interact directly with the legacy application.

Vulnerability Management

Vulnerability Scanning: Legacy applications should be subject to regular vulnerability scanning with an industry standard vulnerability assessment tool, where possible, such as Nessus and Qualys. This should occur on a regular basis, ideally with scans scheduled to occur automatically at some set time interval. Where appropriate, some vulnerabilities might also be identified using code scanning tools, such as a SAST (Static Application Security Testing) tool to check the codebase for obvious vulnerabilities or SCA (Software Composition Analysis) tool identify vulnerable dependencies used by the application. In some cases none of the above options will be viable for the application and, in this case, direct human assessment of host configuration and manual code reviews might be the only suitable option for assessing the security posture of the legacy application.

Patch Management: Where possible, apply patches raised by the tools described above. Patching efforts should be prioritized on the basis of the severity of the vulnerability and whether the vulnerability has a published CVE (Common Vulnerabilities and Exposures) and/or a publicly listed exploit. In circumstances where patching is not practically possible for the legacy application, consider applying additional restrictions to the application/affected components as noted in the section on Authentication/Authorization.

Data Storage

Confirm that, where ever possible, data handled by the application is both encrypted at rest (i.e. when stored in a database) and in transit. Cheat Sheets on [Cryptographic Storage](#) and [HTTP Strict Transport Security](#) may provide some useful further context. In some circumstances legacy applications might be restricted to the use of older network protocols that only support transmission of data in plain text. In this case it is especially important to apply the most restrictive network access controls possible to the application. This could necessitate temporary or permanent air-gapping (functional isolation) of the application.

Ensuring Maintainability

Where possible, aim to maintain a high degree of institutional expertise regarding the application, so that staff can both remediate security vulnerabilities and troubleshoot the application as needed to ensure business continuity. The following recommendations apply:

- More than one staff member should be adequately trained to troubleshoot/reconfigure the legacy application. This will reduce the risk of complete loss of maintenance capability for the legacy application if one trained member of staff leaves the organization.
- Encourage staff to regularly document processes including recording troubleshoot guides for common failure scenarios for the legacy application.
- It might be necessary to teach a core group of staff to write basic programs in the language used by the legacy application, where this expertise does not exist in your organization.

Change Management

The ultimate goal for most legacy applications will be to migrate from the unmaintainable system to a solution which is both maintainable and architected to be resilient to contemporary threats. Staged change management may take into account the following factors:

- What budget can practically be allocated for upgrading to a modern solution and within what time frame could budget be made available?

- Do people with the necessary expertise required to handle migration exist within your organization or could these people be acquired/developed?
- How urgently does a migration to an upgraded solution need to happen based on the risk profile of the application and the risk appetite of your organization?

A change management plan, formal or informal, should include a clear description of granular steps to be taken towards migration to an upgraded solution, an explicit date of expected completion, and a clear articulation of the business and security case for the change. To produce a realistic plan for migration, staff involved in overseeing and using the existing solution should be consulted extensively to get a sense for how critical the legacy application is to your organization and what barriers there might be to facilitating migration or completely decommissioning the application.

Continuous Monitoring and Incident Response

Legacy applications should be subject to an especially high degree of security monitoring with rapid response efforts made to investigate potential incidents. This might be challenged by intra-operability issues that mean that logs produced by the application are in a format that cannot be readily ingested by security monitoring tools used by your organization. Potential workarounds might include:

- Developing custom APIs for modifying security-applicable information from your legacy application and/or its logs into a format ingestible by security monitoring solutions used by your organization.
- Where the above is not possible, consider using automation scripts to generate reports that assess for indicators of compromise.
- Be vigilant to any anomalous network traffic into and out of the legacy application environment and to any surges in network activity.
- If you have access to an internal or hired incident response team, ensure that they are aware that incident response and investigation of unusual events should be prioritized for critical legacy systems. Processes for handling application downtime and compromise ideally are to be documented in advance as a part of an incident response playbook. This needs to give staff a clear rundown of emergency procedures including escalation contacts and details of incident response leaders.
- Incident response planning should occur within the broader context of a business continuity plan.